

Bonus Exercise – Fast Fractal Fun with Cython & GPUs

This exercise explores performance optimization techniques for computing the Mandelbrot set.

```
In [1]: import numpy as numpy
import plotly.express as px
import time

from mandelbrot import mandelbrot_set as mandelbrot_set_py
from mandelbrot_cython import mandelbrot_set as mandelbrot_set_cy
from mandelbrot_gpu import mandelbrot_set as mandelbrot_set_gpu
```

```
In [2]: # Parameters for the Mandelbrot Set
width,height = 1000,800
x_min,x_max,y_min,y_max = -2,1,-1,1
```

Original Code

```
import numpy as np
import matplotlib.pyplot as plt

def mandelbrot(c, max_iter=100):
    """Computes the number of iterations before divergence."""
    z = 0
    for n in range(max_iter):
        if abs(z) > 2:
            return n
        z = z*z + c
    return max_iter

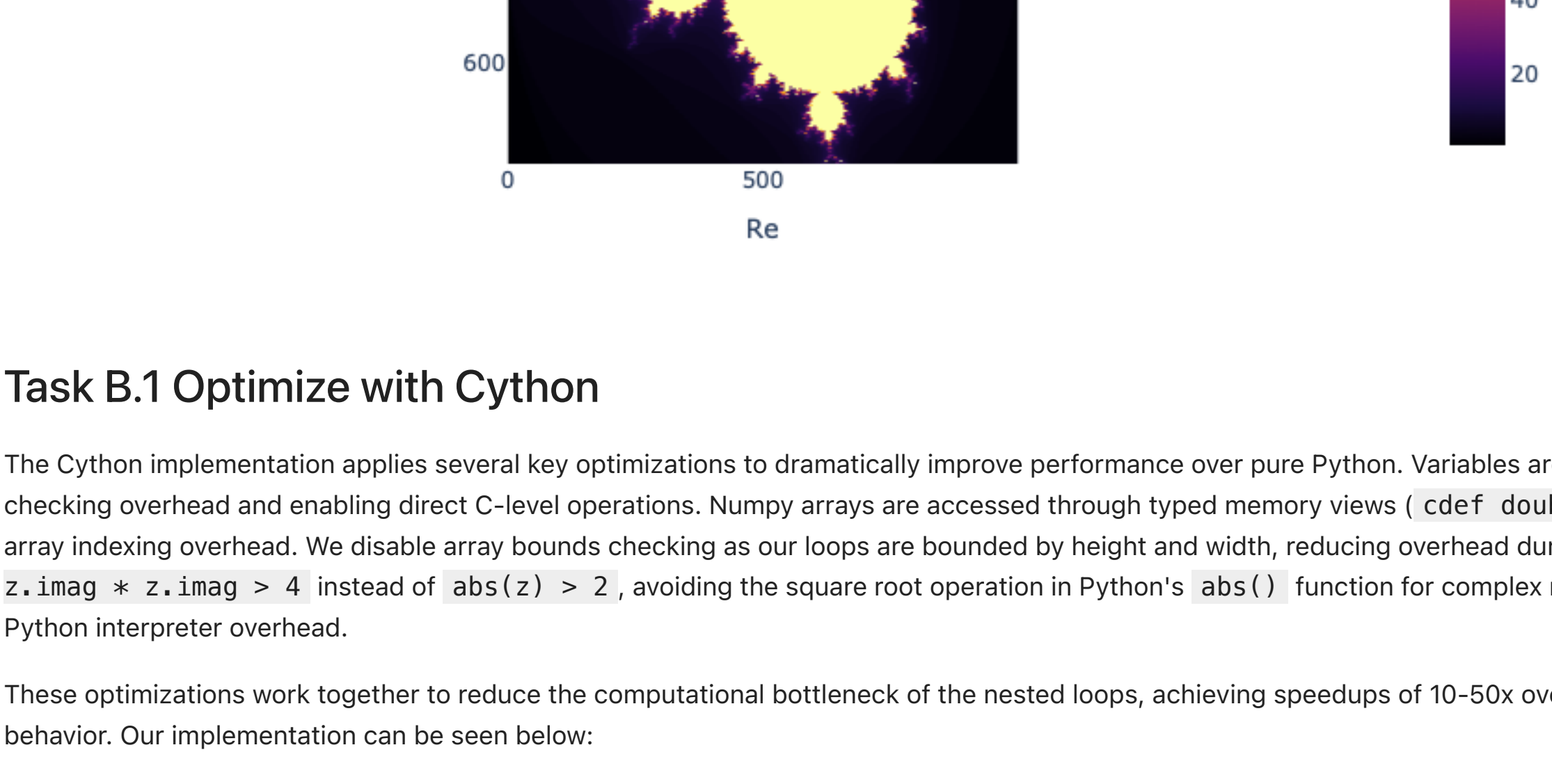
def mandelbrot_set(width,height,x_min,x_max,y_min,y_max,max_iter=100):
    """Generates the Mandelbrot set image."""
    x_vals = np.linspace(x_min,x_max,width)
    y_vals = np.linspace(y_min,y_max,height)
    image = np.zeros((height,width))

    for i in range(height):
        for j in range(width):
            c = complex(x_vals[j],y_vals[i])
            image[i,j] = mandelbrot(c,max_iter)

    return image
```

```
In [3]: python_image = mandelbrot_set_py(width,height,x_min,x_max,y_min,y_max)
```

```
In [4]: # plot the result
fig = px.imshow(python_image,color_continuous_scale='inferno')
fig.update_layout(title='Mandelbrot Set (Python)',xaxis_title='Re',yaxis_title='Im')
fig.show()
```



Task B.1 Optimize with Cython

The Cython implementation applies several key optimizations to dramatically improve performance over pure Python. Variables are declared with explicit C types, eliminating python's dynamic type checking overhead and enabling direct C-level operations. Numpy arrays are accessed through typed memory views (`cdef double[*], :]`) which provides quick direct memory access without Python's array indexing overhead. We disable array bounds checking as our loops are bounded by height and width, reducing overhead during array access. The magnitude check uses `z.real * z.real + z.imag * z.imag > 4` instead of `abs(z) > 2`, avoiding the square root operation in Python's `abs()` function for complex numbers. The entire code set is compiled to optimized C code,eliminating Python interpreter overhead.

These optimizations work together to reduce the computational bottleneck of the nested loops, achieving speedsups of 10-50x over the pure Python implementation while maintaining identical algorithmic behavior. Our implementation can be seen below:

mandelbrot_cython.pyx

```
# cython: boundscheck=False
import numpy as np
cimport numpy as np

def mandelbrot(double complex c, int max_iter=100):
    cdef double complex z = 0
    cdef int n

    for n in range(max_iter):
        if (z.real * z.real + z.imag * z.imag) > 4:
            return n
        z = z * z + c
    return max_iter

def mandelbrot_set(int width, int height, double x_min, double x_max, double y_min, double y_max, int max_iter=100):
    cdef double[:, :] image = np.zeros((height,width), dtype=np.float64)
    cdef double[:, :] x_vals = np.linspace(x_min,x_max,width)
    cdef double[:, :] y_vals = np.linspace(y_min,y_max,height)

    cdef int i, j
    cdef double complex c

    for i in range(height):
        for j in range(width):
            c = complex(x_vals[j],y_vals[i])
            image[i,j] = mandelbrot(c,max_iter)

    return image
```

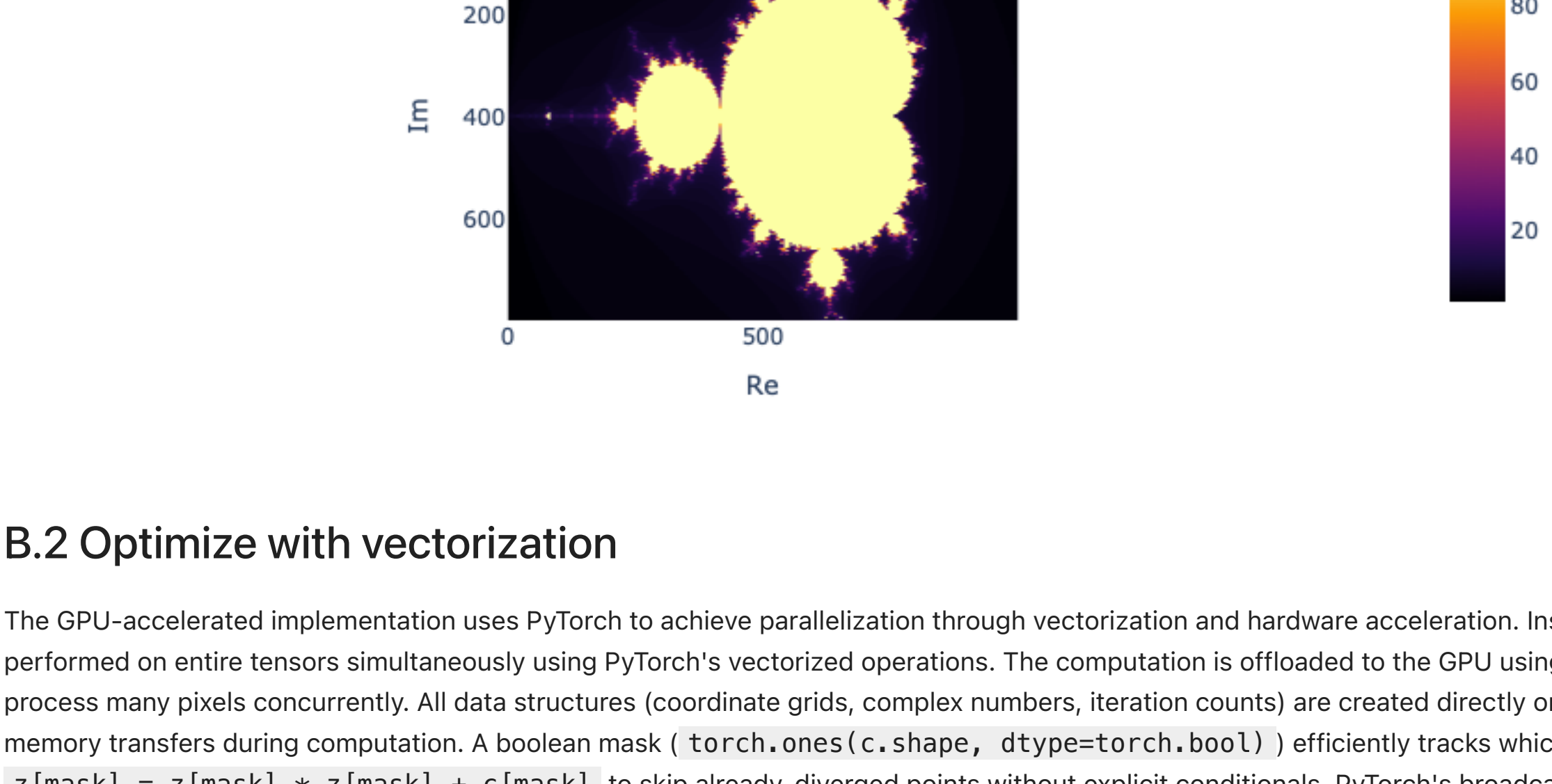
Setup.py

```
from setuptools import setup
from Cython.Build import cythonize
import numpy

setup(
    ext_modules=cythonize("mandelbrot_cython.pyx", compiler_directives={'language_level': "3"}),
    include_dirs=[numpy.get_include()]
)
```

```
In [5]: cython_image = mandelbrot_set_cy(width,height,x_min,x_max,y_min,y_max)
```

```
In [6]: # Plot the Cython result
fig = px.imshow(cython_image,color_continuous_scale='inferno')
fig.update_layout(title='Mandelbrot Set (Cython)',xaxis_title='Re',yaxis_title='Im')
fig.show()
```



B.2 Optimize with vectorization

The GPU-accelerated implementation uses PyTorch to achieve parallelization through vectorization and hardware acceleration. Instead of nested loops processing pixels sequentially, operations are performed on entire tensors simultaneously using PyTorch's vectorized operations. The computation is offloaded to the GPU using `torch.device("cuda")`, where thousands of CUDA cores can process many pixels concurrently. All data structures (coordinate grids, complex numbers, iteration counts) are created directly on the GPU using `device=device`, eliminating expensive CPU-GPU memory transfers during computation. A boolean mask (`torch.ones(c.shape, dtype=torch.bool)`) efficiently tracks which pixels are still being computed, allowing masked operations like `z[mask] = z[mask] * z[mask] + c[mask]` to skip already-diverged points without explicit conditionals. PyTorch's broadcasting and optimized CUDA kernels handle the underlying parallel execution, maximizing GPU utilization across the entire image simultaneously.

These optimizations work together to eliminate the sequential bottleneck of nested loops through massive parallelism. The implementation also supports CPU fallback (`use_gpu=False`) which still benefits from vectorization, though without GPU-level parallelism. Our implementation can be seen below:

```
import torch

def mandelbrot_set(width,height,x_min,x_max,y_min,y_max,max_iter=100,use_gpu=True):
    device = torch.device("cuda" if torch.cuda.is_available() and use_gpu else "cpu")

    x = torch.linspace(x_min,x_max,width,device=device)
    y = torch.linspace(y_min,y_max,height,device=device)
    y_grid,x_grid = torch.meshgrid(y,x,indexing='ij')

    c = torch.complex(x_grid,y_grid)
    z = torch.zeros_like(c)

    image = torch.zeros(c.shape, dtype=torch.float32, device=device)
    mask = torch.ones(c.shape, dtype=torch.bool, device=device)

    for n in range(max_iter):
        z[mask] = z[mask] * z[mask] + c[mask]
        diverged = torch.abs(z) > 2
        new_divergences = diverged & mask
        image[new_divergences] = n

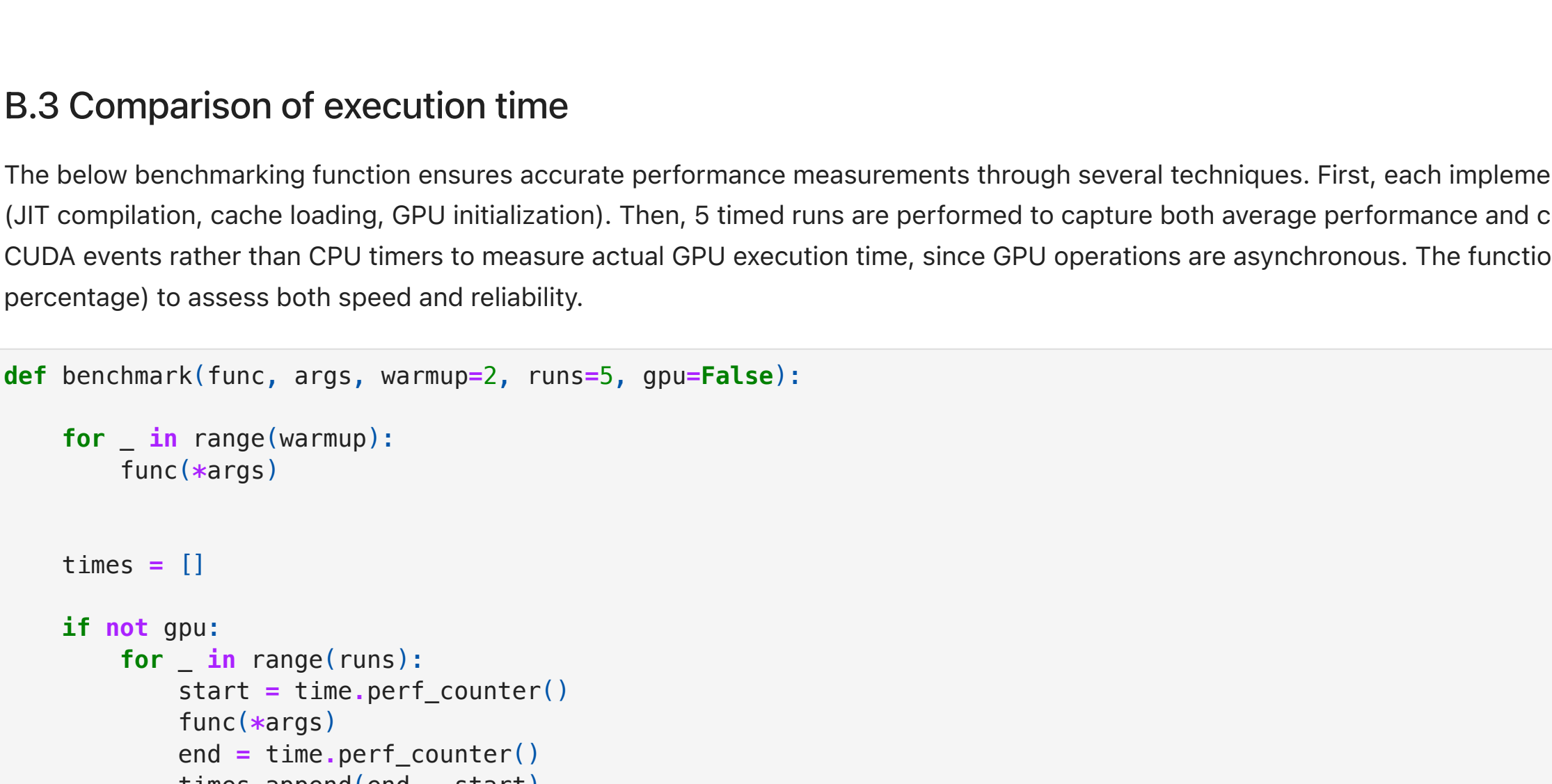
        mask &= ~diverged

    image[mask] = max_iter

    return image.cpu().numpy()
```

```
In [7]: gpu_image = mandelbrot_set_gpu(width,height,x_min,x_max,y_min,y_max)
```

```
In [8]: fig = px.imshow(gpu_image,color_continuous_scale='inferno')
fig.update_layout(title='Mandelbrot Set (GPU)',xaxis_title='Re',yaxis_title='Im')
fig.show()
```



B.3 Comparison of execution time

The below benchmarking function ensures accurate performance measurements through several techniques. First, each implementation runs 2 warmup iterations to eliminate overhead from initial execution (JIT compilation, cache loading, GPU initialization). Then, 5 timed runs are performed to capture both average performance and consistency, accounting for system variability. GPU implementations use CUDA events rather than CPU timers to measure actual GPU execution time, since GPU operations are asynchronous. The function returns average execution time and deviation (max-min difference as a percentage) to assess both speed and reliability.

```
In [ ]: def benchmark(func, args, warmup=2, runs=5, gpu=False):
    for _ in range(warmup):
        func(*args)

    times = []

    if not gpu:
        for _ in range(runs):
            start = time.perf_counter()
            func(*args)
            end = time.perf_counter()
            times.append(end - start)
    else:
        # Measure GPU time using CUDA events
        import torch
        current_stream = torch.cuda.current_stream()
        start_event = torch.cuda.Event(enable_timing=True)
        end_event = torch.cuda.Event(enable_timing=True)
        for _ in range(runs):
            start_event.record(stream=current_stream)
            func(*args)
            end_event.record(stream=current_stream)
            end_event.synchronize() # Wait for the event to complete
            times.append(start_event.elapsed_time(end_event) / 1000) # Convert ms to seconds

    avg_time = sum(times) / len(times)
    deviation = (max(times) - min(times)) / avg_time * 100
    return avg_time, deviation
```

We test the different implementations across a range of image sizes (250×250 to 2500×2500 pixels) in order to see how each implementation scales with workload, identifying which optimizations provide the best performance for larger problems.

```
In [11]: # Benchmark with increasing load levels
sizes = [250, 500, 1000, 1500, 2000, 2500]
x_min,x_max,y_min,y_max = -2,1,-1,1

python_times = []
cython_times = []
gpu_times = []
cpu_times = []
python_devs = []
cython_devs = []
gpu_devs = []
cpu_devs = []

for size in sizes:
    width = height = size
    print(f"~ Benchmarking at size {size}x{size}...")

    # 1. Python
    print("\n - Python...")
    py_time,py_dev = benchmark(mandelbrot_set_py, (width,height,x_min,x_max,y_min,y_max))
    python_times.append(py_time)
    python_devs.append(py_dev)

    # 2. Cython
    print("\n - Cython...")
    cy_time,cy_dev = benchmark(mandelbrot_set_cy, (width,height,x_min,x_max,y_min,y_max))
    cython_times.append(cy_time)
    cython_devs.append(cy_dev)

    # 3. Vectorised - GPU
    print("\n - GPU Vectorised...")
    gpu_time,gpu_dev = benchmark(mandelbrot_set_gpu, (width,height,x_min,x_max,y_min,y_max,100,True), gpu=True)
    gpu_times.append(gpu_time)
    gpu_devs.append(gpu_dev)

    # 4. Vectorised - CPU
    print("\n - CPU Vectorised...")
    cpu_time,cpu_dev = benchmark(mandelbrot_set_gpu, (width,height,x_min,x_max,y_min,y_max,100,False), gpu=False)
    cpu_times.append(cpu_time)
    cpu_devs.append(cpu_dev)

    print(f"~ Results: Python={py_time:.3f}s, Cython={cy_time:.3f}s, GPU={gpu_time:.3f}s, CPU={cpu_time:.3f}s")
```

Benchmarking at size 250x250...

- Python...
- Cython...
- GPU Vectorised...
- CPU Vectorised...

Results: Python=0.236s, Cython=0.013s, GPU=0.097s, CPU=0.100s

Benchmarking at size 500x500...

- Python...
- Cython...
- GPU Vectorised...
- CPU Vectorised...

Results: Python=0.965s, Cython=0.053s, GPU=0.100s, CPU=0.189s

Benchmarking at size 1000x1000...

- Python...
- Cython...
- GPU Vectorised...
- CPU Vectorised...

Results: Python=3.819s, Cython=0.215s, GPU=0.107s, CPU=0.933s

Benchmarking at size 1500x1500...

- Python...
- Cython...
- GPU Vectorised...
- CPU Vectorised...

Results: Python=0.679s, Cython=0.475s, GPU=0.160s, CPU=2.120s

Benchmarking at size 2000x2000...

- Python...
- Cython...
- GPU Vectorised...
- CPU Vectorised...

Results: Python=15.577s, Cython=0.819s, GPU=0.254s, CPU=5.023s

Benchmarking at size 2500x2500...

- Python...
- Cython...
- GPU Vectorised...
- CPU Vectorised...

Results: Python=24.310s, Cython=1.383s, GPU=0.379s, CPU=6.916s

```
In [13]: # Benchmark showing performance scaling with increasing load
import plotly.graph_objects as go

# Define sizes if not already defined
sizes = [250, 500, 1000, 1500, 2000, 2500]

fig = go.Figure()

# Calculate absolute error values from percentage deviations
python_errors = [python_times[i] * python_devs[i] / 100 for i in range(len(sizes))]
cython_errors = [cython_times[i] * cython_devs[i] / 100 for i in range(len(sizes))]
gpu_errors = [gpu_times[i] * gpu_devs[i] / 100 for i in range(len(sizes))]
cpu_errors = [cpu_times[i] * cpu_devs[i] / 100 for i in range(len(sizes))]

fig.add_trace(go.Scatter(
    x=sizes, y=python_times,
    mode='lines+markers',
    name='Python',
    error_y=dict(type='data', array=python_errors)
))

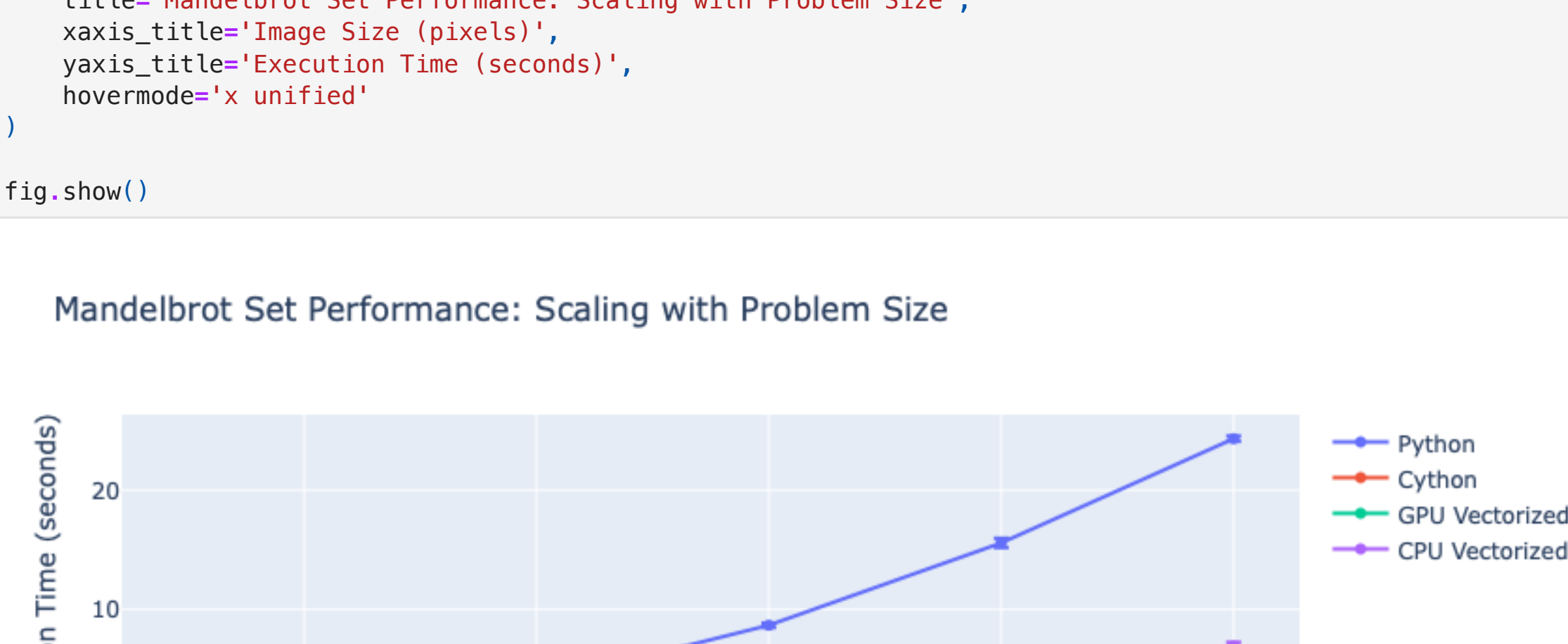
fig.add_trace(go.Scatter(
    x=sizes, y=cython_times,
    mode='lines+markers',
    name='Cython',
    error_y=dict(type='data', array=cython_errors)
))

fig.add_trace(go.Scatter(
    x=sizes, y=gpu_times,
    mode='lines+markers',
    name='GPU Vectorized',
    error_y=dict(type='data', array=gpu_errors)
))

fig.add_trace(go.Scatter(
    x=sizes, y=cpu_times,
    mode='lines+markers',
    name='CPU Vectorized',
    error_y=dict(type='data', array=cpu_errors)
))

fig.update_layout(
    title='Mandelbrot Set Performance: Scaling with Problem Size',
    xaxis_title='Image Size (pixels)',
    yaxis_title='Execution Time (seconds)',
    hovermode='x unified'
)

fig.show()
```



The benchmark results demonstrate clear performance differences across implementations as problem size increases. Pure Python is the slowest, taking 24.3 seconds for the largest image (2500×2500). Cython provides substantial improvement with an 18x speedup at this size, completing in 1.38 seconds.

The GPU vectorized implementation shows the best performance and scalability, maintaining consistently low execution times (0.097s to 0.379s) even as image size increases. This near-constant performance reflects the GPU's massive parallelism, adding more pixels simply utilizes more of the available cores without significantly impacting runtime.

CPU vectorization offers moderate speedups (3.5x faster than Python at 2500×2500) but scales poorly compared to GPU, with execution time growing from 0.100s to 6.916s. This demonstrates that while vectorization helps, true performance gains for large-scale parallel problems require specialized hardware like GPUs.

Interestingly, at the smallest size (250×250), Cython outperforms GPU vectorization (0.013s vs 0.097s) likely due to GPU initialization overhead. However, this advantage disappears quickly as problem size grows, with GPU becoming the clear winner for larger workloads. These results highlight that optimization choice depends on both problem scale and available hardware.